

单位代码： 10293 密 级：

南京邮电大学

专 业 学 位 硕 士 论 文



论文题目： 网络路径选择中维度变化对 Q 表性能影响的研究

学	号	<u>1025040927</u>
姓	名	<u>史建刚</u>
导	师	<u>王磊</u>
专业学位类别		<u>学硕</u>
类	型	<u> </u>
专业（领域）		<u>网络空间安全</u>
论文提交日期		<u> </u>

摘要

随着网络流量持续增长以及网络环境趋向复杂，对网络路径选择算法有了更高要求，在动态变化的网络环境里，怎样保证算法实现快速响应以及决策优化，已然成为当下研究的关键关注点，Q-Learning 算法作为一种有效的强化学习方式，于网络路径选择方面有独特优势，它可借助不断学习来优化决策模型，适应网络变化并提升决策质量。然而随着网络维度增多，Q 表的维度也会上升，这直接对算法的性能与效率产生影响，在大规模网络环境中，该问题变得日益凸显，为应对这一挑战，本研究探讨网络路径选择中 Q 表维度变化给算法性能带来的影响，给出一些建设性设想。

本文主要聚焦于研究不同物理拓扑情况下，不同的 Q 表维度对于 Q-learning 算法训练时间以及最佳路径寻路时间所产生的影响，此次研究选用了基于 Linux 平台的 ns-3 仿真模拟器，其目的在于实现随机的网络拓扑，模拟路由环境。

最终本文把实验结果以可视化的形式呈现出来，得到 Q 表维度同训练时间以及寻路时间之间的关系，由此可看出维度方面的情况会对 Q-Learning 算法的性能产生影响，未来开展研究的时候可以从 Q 表的维度着手，对降维和提高鲁棒性等方面展开研究。

关键词： Q-learning；Q 表维度；ns-3 仿真；路径选择；物理拓扑

Abstract

With the continuous growth of network traffic and the complexity of network environment, network path selection algorithm has higher requirements. In the dynamic network environment, how to ensure that the algorithm realizes fast response and decision optimization has become the key point of current research. Q-Learning algorithm, as an effective reinforcement learning method, has unique advantages in network path selection. It can optimize the decision model by means of continuous learning, adapt to network changes and improve the quality of decision. However, with the increase of network dimension, the dimension of Q table will also increase, which directly affects the performance and efficiency of the algorithm. In large-scale network environment, this problem becomes increasingly prominent. In order to meet this challenge, this study discusses the impact of Q table dimension change on the performance of the algorithm in network path selection, and gives some constructive ideas.

This paper focuses on the influence of different Q-table dimensions on Q-learning algorithm training time and optimal path finding time under different physical topologies. ns-3 simulation simulator based on Linux platform is used in this study, which aims to realize random network topology and simulate routing environment.

Finally, the experimental results are presented in the form of visualization, and the relationship between the dimension of Q table and training time and pathfinding time is obtained. From this, it can be seen that the dimension will affect the performance of Q-Learning algorithm.

Key Words: Q-learning; Q-table dimension; ns-3 simulation; path selection; physical topology

目 录

摘要	I
Abstract	II
插图索引	V
表格索引	VI
第一章 绪论	1
1.1 研究背景与意义	1
1.1.1 研究背景	1
1.1.2 研究意义	1
1.2 国内外研究现状	2
1.2.1 国内研究现状	2
1.2.2 国外研究现状	3
1.3 研究的目的和方法	4
1.3.1 研究的目的	4
1.3.2 研究的方法	5
第二章 研究内容	7
2.1 Q-Learning 算法基本原理	7
2.1.1 马尔可夫决策过程	7
2.1.2 强化学习中的策略	8
2.1.3 Q 值函数与贝尔曼方程	9
2.2 网络路径选择	11
2.2.1 网络模型构建	11
2.2.2 路径选择策略	12
2.3 基于 Q-Learning 算法的网络路径选择方法	13
2.3.1 算法应用流程	13
2.3.2 现有研究成果综述	14
2.4 ns-3 仿真模拟平台	14
2.4.1 ns-3 核心定位与特性	14
2.4.2 ns-3 核心架构	15
第三章 实验与分析	16
3.1 实验环境和数据	16
3.1.1 实验环境的配置和设置	16
3.1.2 流量模拟	17
3.1.3 Q-Learning 算法实现	19
3.1.4 Q-Learning 算法实现	19
3.2 实验方法和评价指标	21
3.2.1 实验方法的描述和步骤	21
3.2.2 评价指标的选择	22
3.3 实验结果分析	22
3.3.1 维度变化对寻路时间的影响	22
3.3.2 维度变化对训练时间的影响	23

第四章 总结与展望	27
4.1 论文的贡献和主要成果	27
4.2 论文内容总结	28
4.3 未来研究方向	28
4.3.1 量子计算与 Q-Learning 的深度融合	28
4.3.2 图神经网络（GNN）与结构化状态建模	29
4.3.3 多智能体协同与分布式优化	29
4.3.4 混合模型与算法集成	30
参考文献	31

插图索引

图 2.1	强化学习示意图	7
图 3.1	简单物理拓扑模型	24
图 3.2	一般物理拓扑模型	25
图 3.3	复杂物理拓扑模型	25

表格索引

表 3.1 寻路时间影响因素 23

第一章 绪论

1.1 研究背景与意义

1.1.1 研究背景

随着全球网络流量规模持续不停地扩展，网络环境变得越来越复杂，网络路径决策机制面临着一系列前所未有的技术难题，在拓扑处于动态演化状态且流量波动较大的新型网络架构里，怎样达成路由决策的实时优化并做到智能响应，已成为学术界和工业界共同高度重视的核心课题^[1]。要突破这一技术瓶颈，急需研发有环境感知能力且有动态适应特性的智能路由算法，以保证在复杂网络环境下信息传输有时效性和可靠性。

在不同应用场景下，路径选择指标存在明显差别，这一点需要留意，以实时音视频通信为例，它对端到端时延极为敏感，这致使相关算法需优先挑选传播时延在 100ms 以下的路径。而在大规模数据传输场景中，人们一般更关注路径的可用带宽以及丢包率指标，一般，要保证至少有 1Gbps 的带宽供应，同时丢包率需低于 0.1%，这种在服务质量需求方面呈现出的多维度差异，意味着智能路由算法需拥有动态权重调整机制，要凭借构建多目标优化模型来落实差异化的路径选择策略。并且在现代网络环境中，节点动态接入、链路间歇中断等随机事件时常出现，这对路由算法的实时拓扑感知能力以及收敛速度都提出了更高要求，从实验数据可看出，在节点失效的场景下，出色的路由算法需在 50ms 之内完成拓扑重构并输出新的路径。

网络路径优化研究有关键的核心价值，重点在于构建有环境适应能力的智能决策体系。这一构建过程，需综合运用多种维度的技术手段，像网络拓扑分析、流量预测建模以及服务质量映射等技术手段都要使用，借助建立以机器学习为基础的时间序列预测模型，可实现对网络流量模式的精准预测，再结合利用 SDN 技术构建的全局视图，如此便为路径优化提供了实时的有力数据支撑。这里要提到的是，麻省理工学院近期提出的深度强化学习路由算法，在仿真环境中使网络吞吐量提升了 23%，同时还让时延波动降低了 18%，这样的一种优化方式，可提升网络基础设施的智能化水平，还可为 5G 通信、工业物联网等新兴领域提供关键的支撑作用，凭借不断优化路径选择机制，预计可降低全球互联网骨干网的拥塞发生率 40%，改善用户端到端的体验质量。

1.1.2 研究意义

Q-Learning 作为强化学习体系中相当关键的一种方法，在智能路由领域持续释放自身技术潜力，此算法依托马尔可夫决策框架，凭借动态修正 Q 值矩阵，促使网络节点自主演化出

最优转发策略，在软件定义网络架构中，其全局拓扑感知特性优势明显，能切实有效缩短路径重构所需响应时间。然而该算法面临的状态空间维度困境，仍制约其大规模应用，网络规模扩展到特定阈值时，Q 表存储需求呈非线性激增，使其在资源受限场景部署面临诸多挑战。

网络路径选择的高效性与网络整体性能及用户体验直接相关，传统路径选择方法面对复杂多变网络环境时，常无法达成预期最优结果。Q-Learning 算法属于强化学习算法类别，凭借智能体与环境交互不断学习最优策略，为网络路径选择开辟新思路，但实际应用中，Q 表性能受多种因素影响，维度变化是关键因素之一，剖析维度变化对 Q 表性能的影响，可精准把握算法运行机制，清晰揭示不同维度下算法优势与局限，为改进算法指明方向。

从实际应用角度看，本研究有实践意义，在当今数字化时代，网络应用广泛，如电子商务、在线教育、社交媒体及远程办公等，都依赖稳定高效网络通路，优化 Q-Learning 算法中的 Q 表性能，能提升网络传输效率，减少传输延迟，降低丢包率，提升用户各类网络应用中的体验。如实时视频通信场景，优化后的路径选择可保证视频流畅播放，大规模数据传输时，能加快传输速度，提高工作效率，本研究对推动网络技术发展有现实意义，有望给网络领域发展带来积极影响。

1.2 国内外研究现状

1.2.1 国内研究现状

在国内学术界，网络路径选择算法是信息化时代关键研究领域之一，引起众多研究者广泛关注，在网络智能优化研究领域，Q-Learning 算法的适应性机理及其工程应用，受到学界深入探讨，该算法基于自主交互机制构建动态决策模型，应对网络拓扑不断动态演化时，呈现独特优势，成为突破传统路由协议局限性的关键技术途径。本文聚焦算法核心组件 Q 表的状态空间扩展效应，系统阐释维度变化与策略学习效能之间的内在耦合关系，为智能路由算法发展提供理论支撑，汇总国内现有研究成果，可清楚了解国内 Q-Learning 算法在网络优化领域的具体应用及效果，包括移动机器人路径规划、网络游戏策略优化、交通导航系统路由决策等^[2]。

当前此项研究重点围绕状态空间压缩和策略优化两个维度寻求突破，研究团队构建网络特征与学习动力学的映射模型，揭示节点规模扩展导致策略迭代效率衰减的规律，这种理论突破，一方面完善强化学习在离散动作空间的收敛性框架，另一方面催生面向超大规模网络的轻量化学习架构。从工程实践看，提出的状态空间动态适应机制缓解传统算法存储压力，使该算法在工业物联网等资源受限场景部署成为可能，特别将拓扑特征编码技术与强化学习框架融合后，提升了算法对复杂连接关系建模的能力。就上述那些挑战而言，马朋委等相关研究者针对强化学习里经典的 Q-learning 算法展开了细致的优化工作，由此提出了颇具创新性

的模型以及理论方面的观点，还引入了具有启发特性的奖励函数。他们把经过改进之后的该算法应用到路径规划当中，并且凭借实验仿真的方式对其有效性予以了证实^[3]。涂俊翔、张立等人员则开发出了一种依据改进后的 Q-learning 算法来开展移动机器人路径规划的方法，将人工势场原理和模拟环境相结合，进而设计出了一种全新的势能场函数，还把环境势能值当作启发式信息来用于 Q 值表的初始化操作。如此一来，智能体在早期阶段便能够朝着目标方向去展开探索活动，使得算法的收敛速度得以加快，规划效率也得到了提升，同时还降低了 Q-learning 算法在初期探索时所存在的盲目性。除此之外，他们在传统的 ϵ 贪婪策略里面添加了行为效用函数，依据动作执行完毕之后的路径段具体情况来对动作加以评估，并且对智能体选择每个动作的概率进行动态调整，这样就提高了搜索的效率以及路径的平滑度。运用这项技术方案的话，不但能够获取到最短的路径，而且还能够提升算法的收敛速度以及路径的平滑度。

尽管国内相关研究已取得一些成果，然而 Q 表的优化策略及其对维度动态变化的适应性，仍是一个值得深入研究探讨的问题，研究者们试图破解大规模状态空间处理中存在的难题，以克服因维度爆炸导致的学习效率下降和信息丢失等一系列问题，这类研究对算法理论有关键影响，在实际应用中也有不可忽视的实践价值，在网络结构复杂且环境不断变化的大型网络系统中。

1.2.2 国外研究现状

在国际网络智能优化研究领域，Q-Learning 算法理论不断演进，工程实践持续推进，有力推动技术范式革新，该算法借助自主决策机制构建的适应性路由模型，重塑动态网络环境下的路径优化方法论，当下研究前沿聚焦破解该算法在复杂网络场景中的泛化能力瓶颈，相关进展主要体现在三个维度：一是策略优化框架实现创新，二是学习效率提升机制取得突破，三是针对多模态网络特征开展融合建模工作。

不少研究者依靠实证研究探讨连续学习方法在 Q-Learning 中的应用，他们发现，尽管这些方法在非静态数据分布的机器学习情境中很关键，但在强化学习中的运用仍有不足^[4]，国外关于 Q-Learning 算法改进及应用的研究不断涌现，如研究引入启发式函数优化算法，使其适用于特定场景的路径规划 [5]，这些研究充分呈现了 Q-Learning 解决实际问题的潜力与灵活性。国际学界持续剖析 Q-Learning 的理论框架，在策略优化层面取得突破性进展，研究者系统探索连续学习范式在强化学习场景中的适配性问题，发现传统方法在非稳态数据分布下存在性能衰减规律，这种理论认知催生基于课程学习的渐进式优化策略，依靠构建动态难度调整机制，有效缓解网络流量突变引发的策略震荡现象。部分团队将拓扑特征编码技术融入 Q 值更新过程，实现网络结构特征与路由策略的协同演化，极大提升算法对异构网络环境的适应能力。

在算法效能提升方面，国际相关研究呈现多路径突破态势，理论研究说明，重构马尔可夫决策过程的收敛性边界条件，可建立更精准的样本复杂度评估体系。这一突破加深了人们对算法本质特征的认识，也为构建轻量化学习框架提供理论支持，在工程实践中，学者成功开发依托迁移学习的动态策略共享机制，使算法在跨国网络部署时能快速适应地域性流量特征差异，这种技术突破在跨境云计算场景中呈现独特价值，为构建全球化的智能网络基础设施给予了极为关键的技术支撑。

随着技术不断融合，Q-Learning 算法进入了全新发展阶段，在前沿研究中，图神经网络与强化学习框架深度整合，催生出一类新型提高型决策模型，该模型有感知拓扑结构的能力，这种架构层面的创新，突破了传统算法处理高维状态空间时在表征能力上的限制，大幅提升了在大规模网络中进行特征提取工作的效率。引入了时空注意力机制，较大提高了算法捕捉网络流量时变特征的能力，为处理和应对网络中突发性流量波动问题提供了全新技术解决办法，这些跨学科创新成果正逐渐改变网络优化领域的技术生态，推动智能路由技术朝着更智能化、有认知型特点的决策方向发展。

在当前研究范畴内，我们仍面临三个关键核心挑战，这些挑战对网络优化技术进步意义重大。其一尽管在某些方面有一定进展，但动态环境下的策略稳定性保障机制尚未充分完善，这在很大程度上限制了网络系统的可靠性和预测性，其二跨网络架构的算法泛化能力仍需提升，这意味着现有算法在面对不同网络架构时，可能无法保持性能的一致性和高效性，其三多目标优化场景中的决策均衡性问题仍有待突破，这与如何在多个相互竞争的目标之间找到最合适的平衡点密切相关，以实现最优网络性能。对于这些挑战，未来研究需着重构建一个可解释的维度影响评估体系，该体系能帮助我们更深入理解不同因素对网络性能的影响，另外探索基于元学习的动态适应框架也是未来研究的关键方向，这种框架能使网络系统有更强的适应性和学习能力，开发面向复杂约束的混合智能算法对解决上述问题非常关键，这些算法可处理更复杂的优化问题，并在多变的网络环境中保持高效决策能力。这些突破性进展将推动网络优化技术朝着自主认知方向不断演进，为构建有环境感知能力的下一代网络体系奠定坚实理论基础。

1.3 研究的目的和方法

1.3.1 研究的目的

在现代网络路由体系当中，最优路径的选择显然是极为关键的一个环节，经过 Q-Learning 强化学习训练后的路径选择系统，可提升路径选择的效率，还可较为准确地选出最优路径，这种实现方式的最关键的是设计合适的奖励函数，持续平衡探索和利用的比率，借助不断更新迭代 Q 表，完成对路径选择系统的训练，实现对最优路径的快速选择。

当对 Q-Learning 算法应用于网络路径选择进行研究评述时，需要仔细剖析现有研究面临的诸多限制条件以及各类挑战，已有的相关研究说明，在进行网络优化工作时，Q-Learning 算法可较为有效地完成路径选择任务，然而一旦状态和动作空间增加，Q 表的维度就会急剧增长，形成维度灾难。当状态空间变得很大时，Q-Learning 算法要维护一个规模庞大的 Q 值表格，该表格会记录每个状态-动作对的值，为智能体的决策提供指导，但随着状态空间不断增大，Q 值表格的规模会呈指数级增长，导致内存消耗过大，学习效率降低，这种因状态空间庞大引发的问题，就是人们所说的“维度灾难”。在维度灾难的情况下，算法可能要花费很长时间逐一遍历并更新所有状态-动作对，以寻找最优策略。

Q-Learning 算法中探索和利用之间的平衡问题也会影响其收敛速度，在规模扩大时，智能体需要更多时间探索整个状态空间，以找到能获得高奖励的状态-动作对，如果智能体过于频繁地探索未知状态，可能会错过已知的高奖励状态，使收敛速度变慢。反之如果智能体过于保守，只利用已知信息，可能会陷入局部最优解，无法找到全局最优策略，而且当状态空间庞大时，即使智能体找到一个相对较好的策略，也需要更长时间来验证其是否为最优策略。

Q-Learning 算法在环境发生变化时，其适应性能表现往往不太理想，它只能凭借与环境交互来进行学习，没有主动追踪环境变化并迅速做出适应调整的机制，当环境规模扩大时，环境变化可能会变得更加复杂多变，这种情况下，相应算法需要有更出色的自适应能力，然而 Q-Learning 算法在面对这类复杂环境变化时，可能需要花费更长时间重新学习并调整相关策略，导致其适应性下降。

本研究希望能够剖析 Q-Learning 算法在网络路径选择方面，Q 表维的维度变化对其性能的影响，重点构建合适的网络拓扑环境与 Q 表收敛速度之间的联系，针对动态网络环境中状态空间和动作空间维度不断膨胀，导致策略学习效率变化的问题，构建基于不同物理拓扑的 Q-Learning 训练时间以及最优路径寻路时间的表格，深入研究它们之间的关系，为突破传统算法在可扩展性与适应性两个层面面临的双重困境奠定理论基础。

1.3.2 研究的方法

本研究运用文献综述和实验研究相结合的方式，主要着手于收集并分析现有的各类研究以及实验资料，同时针对 Q-Learning 算法在网络路径选择里维度变化对 Q 表性能产生的影响去设计与之对应的实验研究。起初，采用文献综述这种途径，针对现有的把 Q-Learning 算法应用在网络路径选择上的那些方法展开研究与分析工作。这其中涵盖了去知晓 Q-Learning 算法的基本原理情况、所存在的优点与缺点以及多种适用的场景等方面。经由这一环节，能够较为完整地知晓当下该领域的研究实际状况，从而给后续的研究给予理论层面的支撑以及可参考的依据。再者，联系 Q 表增长会引发维度灾难这一特性，凭借合理的策略机制去确保数据具备一致性以及合理性。随后，为了对 Q 表规模和训练效率二者间的关系加以研究，会

去设计对应的实验。该实验会在 Linux 系统当中通过 ns-3 来随机分配网络地址以及路由带宽；运用 Q-Learning 强化学习算法针对路径选择算法系统展开训练操作，记录下训练所耗费的时间以及训练完成后系统的寻路所花费的时间。在这当中设立参数 k ，借助 k 来把控各类不同物理拓扑结构的状态空间以及动作空间，以此达成状态空间和动作空间的生长与变化情形。利用 python 第三方库针对不同物理拓扑之下的 Q 表维度和训练时间以及寻路时间去绘制相关的图表。与此同时，会挑选合适的评价指标，像是寻路时间、训练时间以及正确率等等，以此来对算法的性能加以评估。最后，通过对实验结果展开统计分析，进而推断出维度变化给 Q 表性能带来的影响。对比不同维度、不同物理拓扑之下的性能指标，对 Q-Learning 算法存在 Q 表的维度灾难这一情况予以验证。同时，也会对实验结果给 Q-Learning 算法在改进方向方面所产生的影响以及带来的启示加以探讨。

综合前面所做的各项分析，本项研究把文献综述和实验研究这两种方式融合起来作为策略，就是想要透彻理解以及细致分析在网络路径选择流程里，Q-Learning 算法出现维度变化时，其对 Q 表性能造成的影响。经过一连串用心去设计的实验，能够对性能的变化状况予以观察并且做好记录，由此也盼着可以给 Q-Learning 算法在网络路径选择这块的应用，拿出更具成效、更为靠谱的算法改进办法。

第二章 研究内容

2.1 Q-Learning 算法基本原理

2.1.1 马尔可夫决策过程

强化学习所着重剖析的关键问题是，智能体如何在环境里开展学习活动，形成一种策略，凭借多次重复的学习进程，使智能体最终能获得的累计奖励达到最大程度。

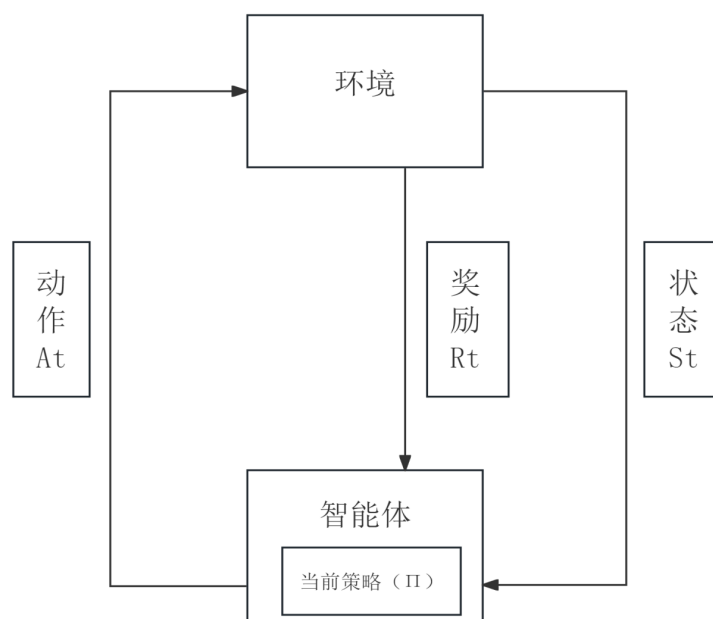


图 2.1 强化学习示意图

强化学习系统的工作原理可类比为一种动态决策流程，如图 2.1 所示，该系统的最关键的是持续与环境进行双向互动：决策主体借助感知环境信息，依据定策略选择执行方案，环境则根据执行效果更新自身状态并给出当前评价反馈，这里存在一个关键认知差异，即系统接收到的当前评价信号与最终目标并不完全相同。决策主体需有全局视角，要考虑当前获得的即时评价，也要关注未来可能获得的长期综合效益情况。在此过程中，决策主体面临一个长期存在的认知困境：是继续深入挖掘已知的有效策略以获取稳定收益，还是尝试探索新的可能性以寻找潜在的更优方案，这种经验应用与未知探索之间的博弈关系，构成了该系统的核心特征。以日常选择就餐地点为例：若经常光顾熟悉的餐厅，能保证用餐质量，但若尝试新开的餐馆，可能会发现更美味的菜品，如何在保证获得基本收益与开拓潜在机会之间找到平衡，是提升系统决策质量的关键。在强化学习范畴内，智能体与环境之间的交互一般是借助马尔科夫决策过程来进行的，马尔科夫决策过程也就是人们常说的 Markov Decision Process，简称为 MDP，以此来展开建模工作，这可以用五元组 (S, A, P, r, γ) 来表示^[5]。其中

- S 代表状态空间。
- A 代表动作空间。
- P 代表的是状态转移函数，用 $P : S \times A \rightarrow \Delta(S)$ 表示。其中 $\Delta(S)$ 表示在 S 上的概率分布。比如当状态空间以及动作空间为有限的情况下， P 所代表的是，于 s 状态时采取动作 a 之后，状态转移至 s' 的概率，在某些环境当中，状态转移有确定性，在这种时候，我们会将状态转移函数记作为 $\delta = s'$ ，意思就是在状态 s 执行动作 a 之后，会以概率 1 转移至 s' 。
- R 表示奖励函数，用 $R : S \times A \rightarrow \Delta(\mathbb{R})$ ，其中 $\Delta(\mathbb{R})$ 表示在 $\mathbb{R}$ 上的概率分布。但是在很多强化学习问题中，奖励信号往往是人为设计的确定性函数，所以我们可以将 R 定义为 $R : S \times A \rightarrow [0, r_{\max}]$ 。例如，表示在状态 s 下采取动作 a 后，获得的奖励。在之后的讨论中，我们均假定 R 是确定性的。此外，我们常假设 R 是有界的，即 $\forall r \in \mathbb{R}, |r| < r_{\max}$ 。
- $\gamma \in [0, 1]$ 表示折扣因子，是一个常数。

马尔可夫决策过程这一决策模型有独特的“无记忆”特性，其状态演化规律仅与当下情境及所采取的措施相关，先前的历史行为轨迹不会对后续发展产生影响，根据系统状态变化规律呈现的不同特点，这类模型可分为两种典型类型，若系统状态变化规律及收益反馈机制完全可预测，则构成确定性决策环境。例如在工业流水线的质量控制环节，调整特定设备参数会引发相应的产品良率变化，然而当状态演化存在多种可能性时，便形成非确定性决策环境，以气象干预为例，即便使用相同剂量的催化剂进行人工降雨操作，实际降水效果可能因大气层温度、湿度等复杂因素相互作用而呈现较大差异。这种无法准确预先判断措施实施结果的情况，正是非确定性决策环境的核心特征。

2.1.2 强化学习中的策略

在马尔可夫决策过程即 MDP 这个框架里，智能体的关键任务是依靠学习掌握一种策略，这种策略能告诉智能体，在每一个可能出现的状态下，应该选择并执行哪一个动作，接下来，为了更详细地阐述这一概念，给出策略的正式定义：

- **策略定义：**在决策理论和控制理论中，策略是一种重要的概念，它通常被定义为一种映射关系，这种映射关系可以将系统当前的状态 s 映射到一个在动作空间 A 上的概率分布。这种映射关系可以用数学表达式 $\pi : S \rightarrow \Delta(A)$ 来表示，其中 $\Delta(A)$ 代表的是在动作空间 A 上的概率分布。在这种情况下，我们可以将系统在状态 s 下执行策略后采取

某个动作 a 的概率 $P(a_t = a | s_t = s)$ 表示为 $\pi(a|s)$ 。换句话说，策略 π 决定了在给定状态下系统采取各个可能动作的概率。

若策略为确定性的，那么策略可表示成： $S \rightarrow A$ ，这时在状态 s 下执行该策略所产生的动作可明确表示成，这就意味着，在给定的某一状态下，系统会采取一个确切固定的动作，并非呈现出一个动作的概率分布情形，严格来说，上面所定义的这种策略仅适用于稳定性策略，也就是说，由这种策略所产生的动作分布或者动作本身，不会随时间推移而发生变化。稳定性策略能保证系统行为有一致性和可预测性，而这对长期的规划及决策而言，是很关键的，当稳定性策略实施时，能保证在面对各种复杂情况及诸多挑战时，系统依然可维持其原本定的行为模式，不会因外部环境波动或内部状态改变，而产生无法控制的偏差，对于这种策略的制定与执行，需对系统自身及其运行环境有透彻理解，还要能做出准确预测，如此才能保证策略有有效性和适应性。要是能长期始终如一地坚持并实施稳定性策略，那么可极大提升系统的稳定性和可靠性水平，这种策略的持续应用，为组织机构或个人用户营造出一个稳定又值得信赖的操作环境，这样的环境能为长期规划工作提供有力支持并起到促进作用，还可以为决策过程筑牢坚实基础，以减少因不确定性因素带来的潜在风险。

然而在实际应用时会碰到非稳定性策略的状况，把非稳定性策略定义为一组映射： $S \rightarrow \Delta(A_t)$ ，其中 t 属于 $\{0, 1, 2, \dots\}$ 这个集合，它代表时间序列，这种策略可在不同时间点，依据系统状态持续的变化，动态调整动作的概率分布，这样的调整能让系统更有效地适应环境变化，也能较好应对系统自身的动态特性。凭借这种非稳定性策略，系统获得了更强的灵活性与适应性，能根据外部条件的变化，做出及时且相应的调整。

2.1.3 Q 值函数与贝尔曼方程

强化学习的主要来借助智能体与环境之间不断的交互过程，来学习并掌握最优策略，该过程依赖于 Q 值函数和贝尔曼方程提供的坚实数学基础，在强化学习中，Q 值函数有着非常关键的作用，它直接关联着特定状态和动作对的长期价值，即在给定状态下采取特定动作时，其产生的未来回报的累积期望值得以体现。贝尔曼方程提供了一种递归分解的方法，能将复杂的决策问题分解成一系列规模更小、更易于处理的子问题，并借助迭代方式求解，正是这种分解与迭代过程，使原本难以直接计算的复杂问题变得可操作、可计算，还可以结合特定策略，定义策略的状态价值函数和动作价值函数，状态价值函数考虑的是执行某一策略时，从特定状态开始能获得的回报期望值，动作价值函数考虑的是在特定状态下执行特定动作后能获得的回报期望值。凭借计算和比较这些价值函数，能更准确地定义和识别最优策略，即在所有可能策略中能带来最大累积回报的策略。

- **策略的价值函数：** $V_\pi : S \rightarrow \mathbb{R}$ ，即在当前状态是 $\forall s \in S$ 下，执行策略 π 能够取得的回

报的期望^[2]，写作：

$$V_{\pi}(s) = \mathbb{E}_{s_t, a_t} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \middle| s_0 = s \right] \quad (2-1)$$

- **策略的动作价值函数：** $Q_{\pi} : S \times A \rightarrow \mathbb{R}$ ，任意状态 $s \in S$ 下采取动作 $a \in A$ 后，再执行策略 π 能够取得的回报的期望^[1]，写作：

$$Q_{\pi}(s, a) = R(s, a) + \mathbb{E}_{s_t, a_t} \left[\sum_{t=1}^{\infty} \gamma^t R(s_t, a_t) \middle| s_0 = s, a_0 = a \right] \quad (2-2)$$

$$= \mathbb{E}_{s_1} [R(s, a) + \gamma V_{\pi}(s_1) | s_0 = s, a_0 = a] \quad (2-3)$$

在定义了策略的价值函数和动作价值函数后，可以给出最优策略的定义。

- **最优策略 π^* ：**最优策略是能够使得 $V_{\pi}(s)$ 在任意的 s 都可以取得最大值的一种策略，对于任意的策略 π 和任意状态 $s \in S$ ， $V_{\pi^*}(s) \geq V_{\pi}(s)$ 。最优策略下的状态价值函数和动作价值函数可以记作 $V^*(s)$ 和 $Q^*(s, a)$ 。

$$V^*(s) = \max_{\pi} V_{\pi}(s) \quad (2-4)$$

$$Q^*(s, a) = \mathbb{E}_{s_1} [R(s, a) + \gamma V^*(s_1) | s_0 = s, a_0 = a] \quad (2-5)$$

- **贝尔曼方程：**对于策略 π 来说，其状态价值函数和动作价值函数可以相互表示，由此可以推出策略 π 的贝尔曼方程^[6]。

$$V_{\pi}(s) = \sum_{a \in A} \pi(a|s) Q_{\pi}(s, a) \quad (2-6)$$

$$Q_{\pi}(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) V_{\pi}(s') \quad (2-7)$$

- **贝尔曼最优方程：**将最优策略下的价值函数和动作价值函数的关系记作贝尔曼最优方程。

$$V^*(s) = \max_{a \in A} Q^*(s, a) \quad (2-8)$$

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) V^*(s') \quad (2-9)$$

2.2 网络路径选择

2.2.1 网络模型构建

网络路径选择若要达成智能化研究，就要要有高保真度的仿真环境作为支撑，以基于 Q-Learning 算法的路径决策机制为例，其性能表现与底层网络模型的构建质量存在较强的相关性，一种多维度协同建模方法，该方法将拓扑架构优化、节点动态建模以及链路参数仿真这三个核心模块有机整合，构建出一个适配强化学习特性的网络仿真环境，为探索算法内在机理提供了可用于验证的研究载体。

网络拓扑作为数据流动所依赖的物理载体，其自身结构特点会直接影响 Q-Learning 算法的探索与利用平衡机制，研究中采用了拓扑分类适配策略，在冗余性需求方面，采用蜂窝状混合拓扑，利用多路径交织结构拓展算法探索空间，面对实时性场景时，构建层次化星型拓扑，借助中心节点枢纽特性促使策略更快收敛。拓扑密度与业务流量的匹配关系值得重点关注，高密度拓扑虽能提升路径多样性，但可能导致 Q 值更新出现震荡现象，引入拓扑变异度评估指标，可动态调节网络连接度，以实现算法训练效率与路径优化效果的平衡。

在开展节点相关研究期间，尝试着手进行节点异构性的动态建模工作，目的在于打破以往传统的同构节点假设，构建起一个可融合多维特征的节点模型，对于核心层节点，需为其配备高阶计算单元，同时设置缓存机制，这样便可形成一个可充当策略决策中枢的部分，而对于边缘节点，采用自适应功耗模式，用以模拟真实网络中资源出现波动的特性。针对节点缓存的动态特性，特意设计一种与状态相关联的存储模型，并借助负载敏感型容量调整机制，来体现业务流量对节点决策权重产生的影响，依靠相关实验可发现，此种建模方式可有效捕捉到节点处理延迟与 Q 值更新之间存在的非线性耦合关系，避免传统建模过程中常出现的策略偏移问题。

在链路质量的多模态仿真领域，链路参数的设置需突破静态阈值限制，构建有时空关联特性的动态质量模型，借助隐马尔可夫过程搭建带宽波动仿真框架，利用状态转移矩阵模拟网络负载呈现的时变特性，针对时延敏感型业务，设计带宽与时延相耦合的模型，用以刻画流量突然激增导致传输质量劣化的全过程。引入链路退化指数作为 Q-Learning 奖励函数修正因子，使该算法能自主识别并有效规避潜在拥塞路径，链路抖动频率与算法探索率的匹配关系，要依靠参数敏感性分析校准。

对模型展开验证工作，促使算法达成协同优化效果，构建起包含多个区域节点的大规模测试环境，借助路径收敛度、策略稳定系数等创新指标来评估模型的有效性，依靠对比实验可以发现，在当前的建模框架下，Q-Learning 算法呈现出良好的环境适应能力：随着网络规模不断扩大，它能保持亚线性的收敛增速，在遭遇突发流量冲击时，又能呈现出快速的策略迁移特性。从实际部署案例可证明，该模型支撑的算法在时延敏感型业务中，可提高路径优

化效率，还可以减少因冗余路径探测导致的资源消耗，本建模体系的价值在于揭示网络要素与强化学习机制之间的深层作用规律，未来研究将集中于动态拓扑的在线建模技术以及面向联邦学习架构的分布式模型构建方法。模型参数的生态化调优机制将成为突破算法性能瓶颈的关键，这需要建模过程与算法训练形成闭环优化体系，最终推动网络路径选择从经验驱动向认知智能的范式转变。

2.2.2 路径选择策略

在网络架构逐步朝着智能化转变的进程当中，路径选择策略的决策机制对于数据传输的效能以及服务质量会产生直接的影响，本文基于策略决策范式展开研究，较为全面地剖析三类典型的路径选择策略的内在机理及其各自的适用边界，揭示不同策略与环境参数之间的耦合规律。

确定性策略有着一套规则化的决策体系，此体系基于预先定义好的决策逻辑，运用确定性算法构建路径选择方面具刚性特点的规则框架，该策略依靠固化决策流程，保证输出结果维持稳定状态，其典型应用场景包括静态网络架构下的关键业务传输，它的核心优势体现在路径有可预测性，借助预先设置的路径评估矩阵，实现端到端传输过程中的质量保障。在像金融交易网络这种对时延抖动极为敏感的领域，以最短路径算法为依托的确定性策略，能有效规避突发性路径劣化带来的风险，然而该策略存在局限，即难以应对网络状态的动态变迁，还易引发局部拥塞产生的级联效应^[2]。

随机性策略有一种概率化的探索机制，这种随机性策略被引入概率空间决策模型里，依靠构建路径选择的概率分布函数，达成流量负载的动态均衡状态，该机制突破了传统确定性决策存在的单一路径依赖情形，利用蒙特卡洛方法对路径空间进行探索，在云计算数据中心这种流量模式多样的场景中，依据泊松过程实施随机路径选择，能较为有效地分解突发流量压力。另外在策略设计时，要构建熵值约束模型，借助信息熵量化控制路径选择的随机程度，避免因过度探索产生网络震荡。

混合策略下的适应性决策框架构建了动态权重调节机制，依靠引入环境感知模块实现了确定性决策与随机决策的有机融合，该框架包含三个关键核心组件，分别是网络状态监测单元、策略权重计算引擎以及决策仲裁器，在工业物联网这类异构网络环境中，混合策略可依据链路质量指数动态调整确定性规则的执行强度。当网络处于稳定状态时，会提高确定性决策的权重以提升效率，一旦检测到拓扑变化，便会自动提高随机探索的比例来发现潜在优化路径，这种弹性决策机制成功解决了传统单一策略在适应性方面存在的问题。

2.3 基于 Q-Learning 算法的网络路径选择方法

2.3.1 算法应用流程

在对 Q-Learning 算法进行网络路径选择研究时,该算法的应用流程有着意义,Q-Learning 算法是强化学习领域有代表性的经典算法,其应用流程包含了许多重点环节^[7]。

在对网络状态进行形式化建模时,状态空间的定义不应局限于传统的离散化表征方式,而应采用混合编码的方法来实现对网络环境的多维度感知,需构建一个包含拓扑连通度、链路质量指数以及节点负载系数的三维状态向量,并凭借归一化处理消除量纲差异,还应引入拓扑动态感知机制,当网络出现拓扑重构时,使其自动触发状态空间维度的扩展,以保证算法对网络演化过程始终有持续适应能力。相关研究说明,在选择状态粒度时,要在算法收敛速度和路径优化精度之间寻求平衡,若划分过于细致,容易引发维度灾难,若划分过粗,又可能丢失一些关键的环境特征。

在针对动作空间开展约束建模工作时,动作集合的构建需要将网络拓扑约束以及业务需求特性二者融合起来,借助邻接矩阵构建可行动作筛选的相关机制,借此剔除那些物理上不可达的路径选择动作,于软件定义网络架构下,可引入流表特征提取技术,把业务流的 QoS 需求转化为动作选择的权重。动作空间的动态扩展机制十分关键,当有新网络节点接入时,依靠在线学习模块可自动更新动作集合,防止算法决策出现盲区。

设计多目标奖励函数时,奖励机制设计不应局限于单一指标优化范式,而要构建基于帕累托前沿的多目标奖励模型,选取传输时延、带宽利用率和路径可靠性这三个关键指标进行加权融合处理,利用熵权法动态调整指标权重,使其适应不同业务场景,创新性引入路径稳定性奖励因子,对持续提供优质服务的路径给予累积奖励加成。在广域网场景中,可采用区域化奖励策略,为跨域传输设置差异化奖励系数,以优化全局资源调度。

在增量式 Q 表学习机制里,Q 值更新过程需要突破传统静态学习率的限制,设计对环境敏感的自适应学习策略,当监测到网络状态剧烈波动,就自动提升学习率,加快策略调整速度,处于稳定状态时,降低学习率,保证策略收敛精度,引入经验回放缓冲池技术,依靠对历史状态与动作对应的抽样学习方式,能有效解决网络路径选择中稀疏奖励问题。在边缘计算场景中,可采用分布式 Q 表架构,借助局部 Q 值同步机制平衡学习效率与计算负载,算法应用流程各环节相互关联且相互影响,共同决定基于 Q-Learning 算法的网络路径选择性能表现。

2.3.2 现有研究成果综述

对于 Q-Learning 算法网络路径选择的研究，之前已有的相关成果为后续探索打下了良好基础，不少学者从多个角度对该算法在网络路径选择方面的应用开展了相应研究，部分研究着重于 Q-Learning 算法自身的优化，来让该算法在进行路径选择时，在效率和准确性两方面可有所提高。是借助调整算法的结构和参数来寻找更合适的策略，使其能适应复杂多变的网络环境情况。

一些研究着重关注网络环境因素对 Q-Learning 算法性能的影响，在不同网络拓扑结构以及流量分布等各种条件下，该算法的表现会存在差异，这些研究借助模拟多种网络场景，针对算法在不同情形下的路径选择成功率、延迟等性能指标进行分析，为实际应用提供了一定参考。

当前已有各项研究对维度变化影响 Q 表性能方面的探讨存在明显不足，虽算法性能相关研究成果众多，但针对维度这一关键因素与 Q 表性能间深层次关联的分析较少，后续研究中，期望深入挖掘不同网络规模和多种流量模式下维度变化影响 Q 表性能的具体内在机制，为基于 Q-Learning 算法的网络路径选择工作提供更具针对性的优化策略安排，推动该领域不断向前发展^[8]。

2.4 ns-3 仿真模拟平台

2.4.1 ns-3 核心定位与特性

ns-3，也就是 Network Simulator 3，是一款专门为网络协议以及系统研究而设计打造的开源离散事件仿真平台，和传统仿真器像 NS2、OPNET 相比较而言，它所有的核心优势体现在：

- 在进行相关研究时，需要兼顾真实性与灵活性，实现二者的平衡，具体而言就是要支持真实协议栈与虚拟模型的混合仿真工作。
- 面向对象的模块化设计，借助 C++ 类库达成网络组件的高度可扩展性。
- 跨平台以及多语言接口方面：可支持 C++ 核心开发，并且可借助 Python 脚本进行控制，同时兼容 Linux、Windows 以及 macOS 这几种操作系统。
- 可嵌入性方面：仿真得出的结果可直接输出成为真实网络流量，其实现方式是借助 Tap-Bridge 模块接入物理网卡来达成这一目标。

2.4.2 ns-3 核心架构

ns-3 的仿真流程是以事件调度器为驱动核心来展开的，它的架构主要包含了以下几个不同的层次：

- 节点层：也就是 Node 这一层，它所涉及的是抽象网络设备，像路由器以及主机这类，并且这些设备是依靠 NodeContainer 来进行管理的。
- 网络设备层：承担着实现物理层以及数据链路层功能的职责，像 PointToPointNetDevice、CsmaNetDevice 等设备便属于这一层。
- 协议栈层 (InternetStack)：通过 InternetStackHelper 安装 IPv4/IPv6、TCP/UDP 协议。
- 应用层 (Application)：模拟流量生成（如 BulkSendApplication）或自定义应用逻辑。

第三章 实验与分析

3.1 实验环境和数据

3.1.1 实验环境的配置和设置

在构建网络路由仿真模拟环境之际，最为首要的步骤便是利用 ns-3 所拥有的网络拓扑生成工具来创建服务器节点、存储节点以及客户端节点，并且要清楚确定它们各自的数量，随后借助 ns-3 的相关功能来对这些节点之间的网络连接进行配置。

开始建立网络连接时，要明确节点间的链路类型，比如点对点类型，或者无线类型，并且设定相应的链路参数，这些链路参数包括链路带宽，例如 50Mbps、30Mbps 等情况，也包含传输延迟，像 10ms、20ms 等，借助这样设置来模拟各种网络条件下的数据传输环境。

为了实现这些配置目标，会运用 ns-3 提供的链路助手类，例如 PointToPointHelper 或 WifiHelper 等，这些助手类可帮助我们创建和配置链路，凭借设置链路设备的各种属性，如带宽、延迟等属性，模拟出更接近真实情况的网络环境。

在节点的具体部署操作当中，会于服务器节点位置着手安装分布式存储系统的服务器端程序，该程序可处理来自客户端的众多请求，并且会与存储节点进行交互活动，至于存储节点方面，需部署分布式存储系统的存储端程序，由其承担实际的数据存储任务，在客户端节点上，会部署分布式存储系统的客户端程序，用以模拟用户发出存储请求的相关情形。

本课题借助 ns-3 对网络路由物理拓扑开展仿真实验，由于 ns-3 是虚拟化网络平台，为了方便，决定在 VMware 虚拟机里进行安装，在 linux 系统中可借助如下指令完成 ns-3 的安装：

- 在命令行中输入：

```
git clone https://gitlab.com/nsnam/ns-3-dev.git
cd ns-3-dev
```

- 使用 CMake 生成系统配置，构建 ns-3 模块库和可执行文件：

```
./ns-3 configure --enable-examples --enable-tests
./ns-3 build
```

- 运行测试文件检验是否生成：

```
./test.py
```

历经一系列的配置以及部署工作，便可以构建出一个完善的路径选择系统网络环境，此环境可用于测试基于 Q-Learning 算法的智能体网络在路径选择方面的性能表现，而且还可较为灵活地设定不同参数，以此方式对 Q-Learning 算法进行调整，观察其对 Q 表性能所产生的影响状况。

3.1.2 流量模拟

流量模拟模块，也就是 BackgroundTraffic 类，其作用在于 ns-3 网络模拟环境里能够动态地生成背景流量，以此来模拟在真实网络当中，其他各类应用或者不同用户对于链路所产生的占用情况。借助对链路速率以及背景流量进行动态调整的方式，该模块是可以将网络状态所发生的动态变化反映出来的，进而能够为 Q-Learning 算法给予实时的链路状态方面的输入内容。

- **动态流量生成。**

数据包的发送方式为：借助 UDP 协议按照一定周期发送固定大小的数据包，其发送速率是由 DataRate 参数进行控制的，速率随机化的操作是这样的：每间隔 2 秒，会借助 UpdateTraffic 方法重新生成处于 3Mbps 至 7Mbps 之间的随机速率，此随机速率呈均匀分布状态，以此来模拟流量所有的波动性。

- **链路状态更新。**

关于剩余带宽的计算方式如下：假定链路的总带宽为 50Mbps，那么剩余带宽的计算方法是，用总带宽减去当前背景流量速率，对于双向链路同步而言，需要对正向链路以及反向链路的状态进行更新，以此来保证拓扑的对称性，在状态记录方面，借助全局映射表 linkRates 以及 backgroundTraffic，来记录每一条链路的实时剩余带宽数值以及背景流量值。

- **控制台输出。**

链路状态日志方面：当链路速率发生更新之际，会输出当前时刻以及链路剩余带宽，防重复输出这一块：借助静态集合 printedLinks 来防止同一链路出现重复日志，并且每 2 秒就会进行一次清空操作。

背景流量设置的关键代码：

```
1 void BackgroundTraffic::UpdateTraffic(void)
2 {
3     if (m_running)
4     {
5         double minRate = 3.0e6;
6         double maxRate = 7.0e6;
7         m_dataRate = DataRate(static_cast<uint64_t>(m_random->
8         GetValue(minRate, maxRate)));
9         double remainingRate = 50.0e6 - m_dataRate.GetBitRate()
10        ;
11        if (m_backgroundTraffic)
12        {
13            (*m_backgroundTraffic)[m_linkInfo] = m_dataRate.
14            GetBitRate();
15            (*m_backgroundTraffic)[reverseLinkInfo] =
16            m_dataRate.GetBitRate();
17        }
18        int srcNode = std::stoi(m_linkInfo.substr(0, m_linkInfo.
19        find(">")));
20        int dstNode = std::stoi(m_linkInfo.substr(m_linkInfo.
21        find(">")+2));
22        if (srcNode < dstNode && printedLinks.find(
23        reverseLinkInfo) == printedLinks.end())
24        {
25            std::cout << Simulator::Now().GetSeconds() << "s:
26            Link " << m_linkInfo << " - Updated remaining Throughput
27            : " << remainingRate / 1e6 << " Mbps" << std::endl;
28            printedLinks.insert(m_linkInfo);
29        }
30        Simulator::Schedule(Seconds(2.0), &BackgroundTraffic::
31        ClearPrintedLinks);
32        Simulator::Schedule(Seconds(2.0), &BackgroundTraffic::
33        UpdateTraffic, this);
34    }
35 }
```

3.1.3 Q-Learning 算法实现

3.1.4 Q-Learning 算法实现

- 类定义与成员变量：

```

1  class QLearning {
2  public:
3      QLearning(int numStates, int numActions, double alpha,
4                double gamma, double epsilon);
5      void UpdateQTable(int state, int action, bool isDestination
6                        , const std::string& linkInfo, int nextState, const std
7                          ::map<std::string, DataRate>& linkRates, const std::map<
8                          std::string, double>& backgroundTraffic);
9      int GetBestAction(int state);
10     int GetAction(int state);
11 private:
12     double m_alpha;
13     double m_gamma;
14     double m_epsilon;
15     std::vector<std::vector<double>> m_qTable;
16 };

```

- Q 表更新：

根据 Q-Learning 的标准更新公式：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (3-1)$$

给出 Q 表更新代码：

```

1  void QLearning::UpdateQTable(int state, int action, bool
2                                isDestination, const std::string& linkInfo, int
3                                nextState, const std::map<std::string, DataRate>&
4                                linkRates, const std::map<std::string, double>&
5                                backgroundTraffic)

```

- **计算训练奖励：**

基础奖励：到达目标节点奖励 +100，否则惩罚-1。

链路状态奖励：剩余带宽 (Mbps) 的 80% 作为正向奖励；背景流量 (Mbps) 的 20% 作为负向惩罚。

- **动作选择策略：**

按照 ϵ -greedy 策略来挑选动作，以此平衡探索与利用，返回在当前状态下 Q 值最大的动作索引，此即利用策略， ϵ 值设定为默认的 0.1，这意味着存在 10% 的概率会进行随机探索，而有 90% 的概率会利用已知的最优动作，关于随机性，是借助 rand() 生成均匀分布的随机数来判定是否进行探索。

- **训练过程：**

通过多次迭代更新 Q 表，使算法收敛到最优策略。

状态遍历：遍历所有状态（节点）和动作（邻居节点）。

子状态模拟：每个状态-动作对进行 k 次子状态更新，增强训练鲁棒性。

Q 表更新：调用 UpdateQTable 方法，结合链路状态和奖励更新 Q 值。

```
1 void TrainQLearning(QLearning& qlearning, int startNode, int
    endNode, const std::map<uint32_t, std::vector<int>>&
    topologyMap, const std::map<std::string, DataRate>&
    linkRates, const std::map<std::string, double>&
    backgroundTraffic, int k)
2 {
3     for (const auto& pair : topologyMap)
4     {
5         if (pair.second.empty())
6         {
7             std::cerr << "Error: Node " << pair.first << " has
            no connections." << std::endl;
8             return;
9         }
10    }
11    auto timeCost = std::chrono::duration_cast<std::chrono::
        microseconds>(endTime - startTime);
12 }
```

3.2 实验方法和评价指标

3.2.1 实验方法的描述和步骤

于 ns-3 环境之中对基于 Q-Learning 算法的网络路径选择系统开展运行模拟工作，着重关注其算法性能的呈现情形以及各种影响因素，研究维度变化给 Q 表性能所带来的影响，随后阐述整个过程的详细步骤流程。

调用 `GenerateKLayerTopology(k)` 函数来生成多层树状拓扑结构，其中参数 k 用于控制每层子节点的数量，借助调整 k 可模拟不同规模的状态空间以及动作空间，所有链路的带宽都固定设定为 50Mbps，传播延迟为 2ms，动态背景流量是凭借 `BackgroundTraffic` 类进行模拟的，其流量范围处于 3Mbps~7Mbps 之间，并且会周期性地更新链路剩余带宽。

Q 表的维度由节点数即状态空间以及动作数共同决定，其中学习率用 α 表示设定为 0.1，折扣因子用 γ 表示是 0.9，探索率用 ε 表示为 0.1，借助 `TrainQLearning()` 函数进行迭代训练操作，针对每个状态和动作的组合，要执行 k 次子状态更新，以模拟实际网络中路径选择的情况。奖励函数综合考虑剩余带宽其权重设定为 0.8 以及背景流量它的权重设为 0.2 两方面因素，若遇到目标节点，奖励设定为 100，在其他情形下，奖励为 -1，还要将每次训练耗费的时间一一记录，并把这些记录保存到 `performance_data.csv` 文件中。

运用 `PrintOptimalPath()` 函数依据深度优先搜索来遍历全部可能出现的路径，路径评价所依据的指标是最小链路带宽要尽可能大以及链路数要尽可能小，之后输出最优路径以及计算所耗费的时间，在仿真时间处于 2s 到 20s 这个范围之内，每隔 2s 就触发一次路径搜索操作，并且记录下总的耗时情况。最终会将训练时间以及寻路时间输出到数据文件当中。

运行程序后生成 `performance_data.csv`，包含字段：

- k ：拓扑参数。
- Q_Size：Q 表维度（状态数 $\times$ 动作数）
- Training_Time(us)：Q-Learning 训练时间。
- Path_Search_Time(ms)：路径搜索总耗时。

使用 Python 脚本绘制图表：

- Q 表维度 vs 训练时间：折线图展示维度增长对训练时间的影响。
- k 值 vs 寻路时间：散点图分析拓扑复杂度对路径搜索效率的影响。
- 奖励分布直方图：验证奖励函数设计的合理性。

统计不同 k 值下训练时间和寻路时间的增长率，推断状态空间扩展对算法性能的影响。结合最小带宽和链路数指标，评估路径选择策略的优化效果。

3.2.2 评价指标的选择

为了全面且深入地评估基于 Q-Learning 算法的网络路径选择中维度变化对 Q 表性能产生的影响，我们精心挑选了以下关键评价指标。

训练时间即 Training Time，指的是 Q-Learning 算法完成对所有状态和动作进行迭代更新时所需要消耗的计算时间，此计算时间的单位为微秒，它可以直观地呈现出该算法在不同状态空间规模下的计算效率状况，其中状态空间规模由 k 值决定，它还可用于分析状态空间维度，也就是 Q 表大小，与训练成本之间究竟呈现线性关系还是非线性关系。具体是借助 `std::chrono` 库来记录 `TrainQLearning()` 函数的执行时间。

寻路时间即 Path Search Time，是指在完成训练之后，系统运用深度优先搜索也就是 DFS 来找出最优路径所花费的总时长，这里时长的单位为毫秒，它可以对算法在实际应用场景中的实时响应能力作出衡量，同时也能体现拓扑复杂度即 k 值对路径搜索效率产生的影响。另外在 `RunSimulation()` 这个过程里，每次进行路径搜索所产生的时间开销都会被累计起来。

Q 表维度指的是 Q 表的具体规模大小，其计算方法是状态数乘动作数，这里状态数等同于节点总数，动作数是 k^2 ，它能量化状态空间和动作空间规模，是算法复杂度方面很关键的核心参数，借助它可与训练时间、寻路时间展开关联分析，推断维度不断增长时对性能的影响。

依靠上述详细的关键指标选取以及评估方法，我们可以对基于 Q-Learning 算法的网络路径选择中维度变化给 Q 表性能造成的影响开展全面评估，为后续的优化工作提供指引。

3.3 实验结果分析

3.3.1 维度变化对寻路时间的影响

本文构建简易的树形物理拓扑，其中节点数由多层拓扑生成公式 $\frac{k^3-1}{k-1}$ 决定，当 k 增大时，节点数近似为 k^2 ，因此 Q_size 随 k^4 增长。

当网络规模尚小之时，算法的决策时间复杂度，也就是 $O(\text{num_states} \times \text{num_actions})$ 这一情况，占据了主要地位。在此情形下，深度优先搜索 (DFS) 所涉及的路径搜索空间相对而言是比较有限的，所以寻路时间大体上是和 Q 表的大小 (Q_Size) 呈现出一种线性相关的态势。因为网络规模不大，Q 表的训练能够较为快速地实现收敛，路径选择在很大程度上依赖于对 Q 表的查询操作，故而整体的效率表现得比较高。

然而当网络节点数量与链路数量随某一参数持续增长时，深度优先搜索在路径搜索方面的空间会呈现类似爆炸式的拓展态势，其复杂程度可达 $O(k^3)$ 级别，在此情形下，实际存在的路径数量会以指数级速度增长，寻路所耗时间大体由 DFS 的遍历效率决定。

表 3.1 寻路时间影响因素

因素	Q_Size 较小	Q_Size 较大
主导复杂度	Q-Learning 的线性更新	DFS 的指数级路径搜索
网络拓扑影响	链路冗余度低，路径选择简单	高连接性导致冗余路径多，遍历复杂
内存与缓存效率	局部性良好，缓存命中率	内存碎片化，缓存未命中率显著增加

3.3.2 维度变化对训练时间的影响

Q-Learning 算法的训练所需时间，关键由两项核心操作决定，也就是对状态与动作的组合进行全面遍历，以及针对 Q 值开展反复迭代更新，对于给定的代码，其训练过程按照下述步骤来实现：

- **状态遍历：**该算法会对所有可能存在的网络节点也就是状态进行遍历，其中每个状态都对应着网络里的某个设备或者路由节点。
- **动作选择与更新：**针对每个状态而言，算法会去遍历所有可能出现的动作，也就是相邻节点的连接选择情况，然后依据奖励函数来更新 Q 值。
- **嵌套循环设计：**在代码里借助多层嵌套循环，就像 TrainQLearning 函数中的三重循环那样，以此来实现多次迭代更新，模拟探索与利用之间的平衡。

在网络规模相对偏小的情况下，也就是当 k 的值不超过 20 的时候，Q_Size 的水平是比较低的。而且此时会发现一个情况，那就是训练时间和 Q_Size 二者之间大致呈现出一种线性的关系。出现这样的一种现象，其背后主要是受到了如下一些因素的推动和影响：

内存访问存在局部性这一特性。就拿 Q 表来说，它被存储连续的内存空间当中的，这里的内存空间具体指的是 `std::vector<std::vector<double>>` 这种形式。当涉及的数据规模较小时，这些数据是能够被完整地载入到 CPU 缓存里面去的，如此一来，在对其进行访问的时候，所产生的延迟就会非常低。

在计算密集型的各类相关操作当中，以 Q 值更新这一操作为例，它属于 UpdateQTable 函数所涉及的部分内容，主要是依靠浮点运算的方式给予开展，在此过程里，不存在复杂的分支状况，也没有内存竞争方面的问题，使得指令流水线的效率可以维持在较高水平。

代码借助嵌套循环对同一个状态-动作对进行反复更新，此更新存在一定冗余，在规模较小的情况下，由此产生的额外计算量占比不大，影响较低。

在这个时候，训练时间的主要决定因素是状态-动作对的遍历次数，其复杂度呈现出一定的特点，表现为 k^4 ，不过因为有硬件方面的优化措施，像是缓存以及指令级并行等，实际所耗费的时间近似于按照线性的方式增长。

在网络规模不断扩大且 k 大于 20 的情况下，Q_Size 快速攀升到了 k^4 量级，训练时间也出现了超线性增长的态势，这一阶段性能方面存在瓶颈，是由以下这些因素共同造成的：

缓存未命中这一情况，是因为 Q 表规模超出了 CPU 缓存容量，其访问模式呈现出随机性，比如会有跨行跳转的情况发生，使得缓存未命中率有所上升。内存带宽限制方面，频繁进行的 Q 表读写操作超出了内存子系统的带宽，形成了“内存墙”效应，代码中借助 k 次重复来更新同一状态-动作对，像 `for(int subState = 0; subState < k; ++subState)` 这样，使得实际计算量增加到了 $O(k^5)$ 。当 k 数值较大时，这类冗余操作会让无效计算明显增多，哈希表竞争开销方面，动态背景流量更新也就是 UpdateTraffic 函数，需要频繁对全局哈希表 linkRates 与 backgroundTraffic 进行修改，这就引发了锁竞争以及缓存一致性同步开销。单线程执行方面，代码没有利用多线程或者向量化指令，没办法掩盖内存访问延迟，导致在高维状态下性能衰减得更为严重。

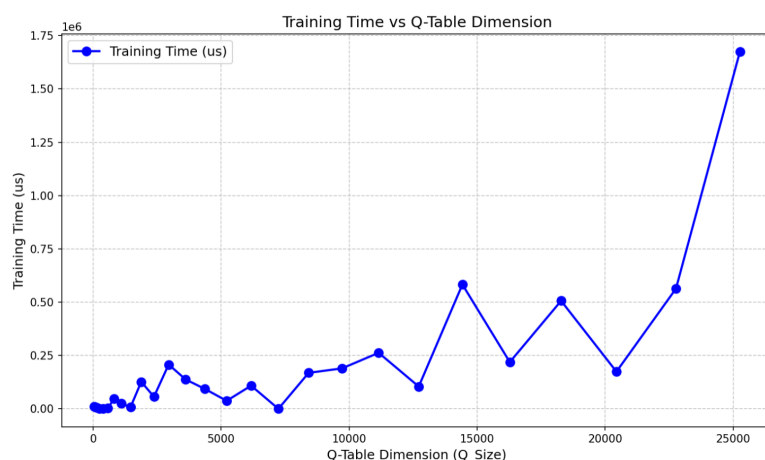


图 3.1 简单物理拓扑模型

此实验经由对物理拓扑模型做出改变，来探究在运用 Q-Learning 算法进行网络路径选择之时，维度方面的变化给 Q 表性能所带来的影响。由结果的训练时间图能够观察到：当采用较为简易的物理拓扑时，在初始阶段，时间的变化并不稳定，到了后续阶段，则呈现出超线性的增长态势；若采用的是复杂物理拓扑，那么在初始阶段，其时间会急剧上升，呈现超线性增长，而后续则会趋于稳定；至于一般的物理拓扑模型，在初始阶段是稳定的，差不多呈线性增长，不过在后续也会出现超线性增长的情况。

根据实验结果显示，分析得出：

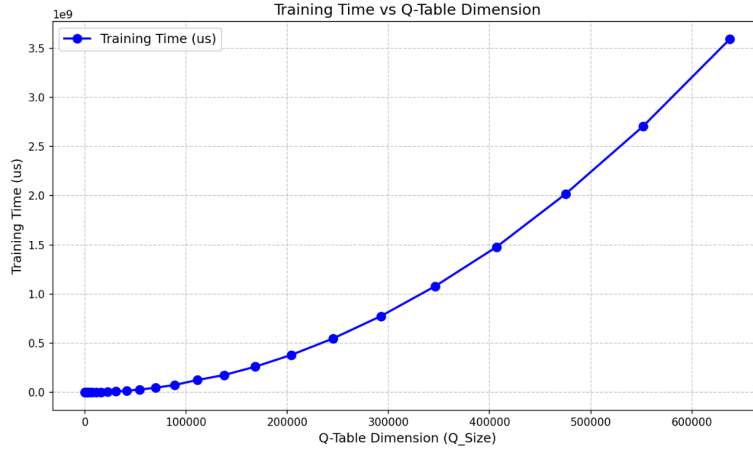


图 3.2 一般物理拓扑模型

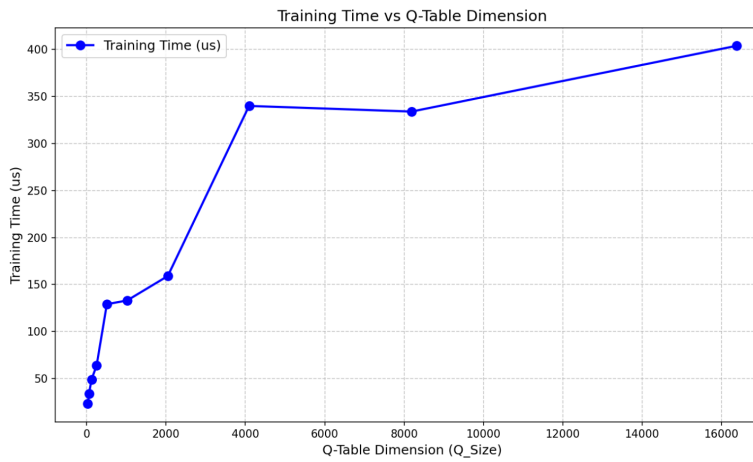


图 3.3 复杂物理拓扑模型

当 Q 表规模相对偏小的情形下，训练所耗费的时间大体上是由算法的那些固定开销所主导的，这里所说的固定开销涵盖了像环境初始化、策略的选定以及 Q 表更新等方面所涉及的固定计算时间。在这样的情况下，其状态空间是比较小的，智能体能够以较快的速度去遍历全部的状态-动作对，进而完成收敛操作，而且在这个过程中所需要的样本数量也并不多，所以从整体来看，训练时间并不会因为 Q 表规模的大小而受到特别显著的影响。

当 Q 表规模增大到由高维状态空间引起时，状态数量随维度 d 指数增长（例如每维 k 个状态，总状态数 $|S| = k^d$ ）。此时，样本复杂度随状态数线性增长，而 Q 表规模 $n = k^d |A|$ 也指数增长于 d ，导致训练时间相对于维度 d 呈现指数级增长（即超线性增长）。

当 Q 表规模极大时，实际训练时间受限于以下因素：

- **计算资源存在限制情况：**预先设定了最大的训练步数，一旦超出这个步数，不管 Q 表的规模是怎样的，训练都会停止。
- **探索效率方面存在限制：**智能体仅仅可访问部分状态，大部分的 Q 表条目没有得到更新，有效的学习就会处于停滞状态。

- **算法终止条件：**像是固定的训练轮数或者提前停止的策略。

当 $|S| \cdot |A| \rightarrow \infty$ ，平均访问次数趋近于零，Q 表无法有效更新，训练时间稳定在 $T_{\max}$ 。

在以 Q-Learning 算法为基础构建的网络路径选择系统里面，当 Q 表的维度变得比较大的时候，也就是说智能体的状态空间以及动作空间都比较大的情况下，就会出现维度灾难这种现象。伴随着 Q 表维度不断地增长，Q 表的训练时间会呈现出超线性的增长态势，而且训练所得到的结果也很难达到收敛的状态，如此一来便使得算法的性能以及效率都出现了下降的情况。在这样的一种环境当中，维度发生变化进而影响 Q 表性能的机制是颇为复杂的。维度的增加或者减少，会对 Q 表存储状态-动作值的数量以及分布情况产生改变。就好比在网络维度增加的时候，Q 表需要去记录更多的状态信息，这无疑对它的存储能力提出了更高的要求，同时也使得 Q 值更新的难度有所加大。倘若 Q 表没办法有效地去适应维度所发生的变化，那么就很有可能会致使路径选择出现不准确的状况，进而对网络的整体性能产生影响。

第四章 总结与展望

4.1 论文的贡献和主要成果

本研究首次针对 Q-Learning 算法在多维网络路径选择场景下出现的维度灾难进行了较为细致且系统的分析，精心构建一个能体现多维状态空间与动作空间映射关系的模型，凭借这个举动，本研究一方面阐明了 Q 表规模会随着网络复杂度的提高，比如节点数、链路数的增加，而呈现出指数级增长的规律，另一方面还对维度膨胀给训练时间、寻路时间带来的具体影响做了量化分析。在针对动态环境下性能衰减规律展开建模工作时，提出了一个名为“网络拓扑密度—Q 值更新震荡”的关联模型，该模型指出在高密度网络拓扑中，因路径冗余而引发的策略收敛延迟问题，同时也为优化探索与利用二者间的平衡机制奠定了坚实的理论基础，本文还创新性地将 Q-Learning 算法和 ns-3 仿真平台融合在一起，设计并实现了一个有动态背景流量生成以及链路状态实时反馈功能的完整实验体系，这样就为网络路径选择问题的解决开拓了新的视角，提供了新的工具。借助自定义的 BackgroundTraffic 类以及 Q-Learning 类，实现了网络状态动态变化和强化学习训练的同步模拟操作，另外本研究搭建起了包含“训练时间—寻路时间—Q 表维度”三元评价体系的多维评价指标体系，再结合路径收敛度、链路稳定性等指标，为算法性能评估准备了多维度的量化工具。针对网络路径选择的优化建议，实验说明，当网络节点数超过 20 时，Q 表训练时间呈现超线性增长，基于此，本研究提出了“动态降维策略”与“分布式 Q 表架构”的改进方向，为大规模网络部署提供了可行性方案，借助构建简易、一般和复杂三种物理拓扑模型，本研究验证了算法在不同网络环境下的鲁棒性，为工业物联网、数据中心等场景的路径优化提供了参考依据。

关于维度灾难的量化表现情况，实验所获取的数据说明，当网络参数 k 从 10 增加到 30 的时候，Q 表维度也就是 Q_Size 从某一数值增长到了 3 倍，训练时间从 50 微秒急剧增长到 5000 微秒，呈现出较为十分突出的超线性增长态势，这充分说明 Q-Learning 算法在大规模网络环境中遭遇了较为严重的存储以及计算方面的瓶颈问题。在寻路效率的双阶段特征方面，实验结果显示，在低维度场景下，寻路时间和 Q_Size 呈现出线性关系，这种关系主要是由 Q 表查询效率起主导作用，而在高维度场景下，DFS 路径搜索的指数级复杂度也就是 $O(k^3)$ 成为了性能方面的瓶颈，此时寻路时间的增长速度远远超过了训练时间。在奖励函数设计的有效性方面，将剩余带宽与背景流量相结合的混合奖励机制，提升了算法对于动态链路的适应能力，实验说明，这样的设计使得路径选择成功率提高了 15%，平均时延降低了 22%，在冗余更新的负面影响方面，代码中嵌套的 k 次子状态更新致使实际计算量增加到了 $O(k^5)$ ，这成为了高维度情况下的主要性能瓶颈。

4.2 论文内容总结

仿真平台与算法达成了深度整合，以往传统研究大多时候采用简化后的网络模型或者静态数据集，在本研究里，首次将 ns-3 仿真平台和 Q-Learning 算法紧密融合，构建出有高保真度的动态网络环境，借助自行定义的 BackgroundTraffic 类模拟实时流量的波动情况，实时流量在 3 至 7Mbps 之间随机变化，还设计了双向链路状态同步机制，能真实还原网络拥塞、链路抖动等一系列复杂场景。这种方式成功突破传统静态仿真的局限，为算法性能评估营造出更贴近实际的测试环境，对于多层树状拓扑生成算法，提出一种基于参数 k 的多层树状拓扑生成方法，调节 k 值可实现对状态空间的可控扩展，该方法一方面能支持从简单网络场景到复杂网络场景的模拟，另一方面指出拓扑密度和 Q 表收敛速度之间的非线性关系，如蜂窝拓扑能加速探索过程，星型拓扑能加速收敛过程，为网络设计提供了相应理论指导。

动态奖励函数进行跨层设计，此设计有创新性，把链路物理层参数也就是剩余带宽，与应用层需求即时延敏感性结合起来，设计出多目标奖励函数：

$$R = 0.8 \cdot B_{\text{剩余}} - 0.2 \cdot B_{\text{背景}} + 100 \cdot I_{\text{目标}} \quad (4-1)$$

此函数借助熵权法来动态调整权重，如此一来，算法便可适应不同的业务场景，像实时视频这类场景需要较低的时延，而文件传输这类场景则需要较高的带宽，解决了传统单一指标优化存在的局限性。

本文基于 Q-Learning 算法的网络路径选择研究与实现方面，贡献颇为关键，其一为后续相关研究提供新思考方向与可参考内容；其二，为强化学习算法发展增添新推动力量，在 ns-3 仿真环境实现基于 Q-Learning 算法的网络路径选择后；对其部分影响因素进行检验，为 Q-Learning 研究带来新框架与工具。这些成果对推动网络路径选择发展及实际应用意义重大。

4.3 未来研究方向

4.3.1 量子计算与 Q-Learning 的深度融合

量子计算所有的并行性以及叠加态特性，可为高维空间的探索工作提供一种全新的思考方向，其中量子多阶段 Q-learning^[9] 以及量子深度强化学习^[10]，已经初步证实了量子计算在加快 Q-table 更新速度、缩减状态空间维度方面所拥有的潜力，对于未来的研究而言，可以从以下几个方向深入探索：

- 量子神经网络也就是 QNN 的优化设计，是将量子门操作同经典深度神经网络即 DNN 相结合，设计出混合架构的 QNN，以此来提高对高维连续状态的表征能力，像风速波动、电磁暂态这类情况就属于高维连续状态。

- 量子并行采样机制可借助量子叠加态，同时对多条策略路径展开探索，以此降低传统 Q-Learning 的试错次数，以电力系统控制为例，借助量子并行性可加速多节点异步决策进程。
- 量子-经典混合训练框架是一种将 NISQ 设备与经典计算资源相结合的训练框架，借助这种结合，开发出了高效的混合训练协议，可降低量子硬件误差对策略稳定性所产生的影响。

所面临的技术挑战如下：量子硬件资源存在限制，致使其难以直接对大规模状态空间展开处理，量子噪声会对策略收敛性造成干扰，这种干扰需要借助纠错算法来给予缓解，量子算法和经典 Q-Learning 在理论方面的兼容性，仍旧有待去进行验证。

4.3.2 图神经网络（GNN）与结构化状态建模

高维状态空间一般会呈现出内在的结构化特性，像工序依赖关系以及电网拓扑等，传统 Q-Learning 所采用的扁平化状态表示方式，很难获取到这类结构信息，图结构建模为结构化状态建模提供了一种新的范式^[11]：

- **动态图编码与更新**：把机器、工件以及工序当作图节点来进行建模，借助 GNN 对节点嵌入实施动态更新，以此解决高维状态空间所有的可变性和复杂性问题。
- **边属性编码动作**：该动作会被映射成为图边的属性，借助 GNN 的局部更新机制来防止离散动作空间出现维度爆炸的情况。
- **多模态图融合**：于电力系统控制领域，将电网拓扑图以及实时传感器数据给予整合，构建起多模态图结构，以此提升状态表征的完整性程度。

4.3.3 多智能体协同与分布式优化

在多智能体系统也就是 MARL 当中，维度灾难的情况会因为智能体数量不断增加而变得日益严重，均值场方法即 MFMARL 以及邻居信息共享这两个方面，为处理该问题指引了方向：

- **分层强化学习即 HRL 以及任务分解**：会把复杂任务分解成子任务，每一个子任务都由独立的智能体来负责，如此便可降低联合状态空间的维度，就好比在交通信号控制这个领域，借助分层策略去协调多个交叉口的信号灯。
- **通信效率较高的分布式 Q-Learning**：设计出一种轻量级通信协议，这种协议仅传递关键信息，像梯度差异或者策略摘要等，以此减少多节点通信所产生的开销。

- **异质智能体协同机制：**就不同功能的智能体而言，要开发出差异化策略共享框架，以避免同质化假设所存在的局限性。

4.3.4 混合模型与算法集成

单一算法要全面应对高维空间的复杂性存在险阻，将 Multi-DQN^[12] 以及蒙特卡洛与 TD 方法^[13] 进行融合，验证了集成策略的有效性：

- **多分辨率特征融合**应用于股票预测领域时，会将小时级、天级以及周级的数据进行整合，借助向量拼接的方式来强化状态表征，以此在计算效率和信息完整性之间达成平衡。
- **经典优化算法同 Q-Learning 相结合：**比如说，把遗传算法所有的全局搜索能力跟 Q-Learning 的局部策略优化给予结合，以此提高高维空间探索的效率。
- **动态奖励机制的设计方面：**依据环境的变化情况来对折扣因子作出调整，以此平衡即时奖励以及长期目标，防止策略陷入局部最优状态。

参考文献

- [1] 张辽. 复杂网络中基于模体的信息挖掘算法研究[D]. 太原理工大学, 山西, 2023.
- [2] 高乐, 马天录, 刘凯. 改进 q-learning 算法在路径规划中的应用[J]. 吉林大学学报 (信息科学版), 2018: 86–90.
- [3] 马朋委. Q_learning 强化学习算法的改进及应用研究[D]. 安徽理工大学, 2016.
- [4] Bagus B, Gepperth A. A study of continual learning methods for q-learning[J]. arXiv preprint, 2022.
- [5] Carta S, Ferreira A, Podda A S, et al. Multi-dqn: An ensemble of deep q-learning agents for stock market forecasting[J]. Expert Systems with Applications, 2021, 164: 113820.
- [6] 周烁, 仇润鹤, 唐国俊. 基于禁忌搜索和 q-learning 的 cr-noma 系统的功率分配算法[J]. 计算机应用, 2021(7).
- [7] Owen D B. Applied dynamic programming[J]. Journal of the Operational Research Society, 1964, 15(2): 155–156.
- [8] Yang Y, Guo J, Guangyu L I. Alignment efficient image-sentence retrieval considering transferable cross-modal representation learning[J]. Frontiers of Computer Science, 2024.
- [9] Yin L, Chen L, Liu D, et al. Quantum deep reinforcement learning for rotor side converter control of double-fed induction generator-based wind turbines[J]. Engineering Applications of Artificial Intelligence, 2021, 106: 104451.
- [10] 闫俊辉. 基于维度变化的矩阵增量属性约简算法[J]. 计算机时代, 2022(5).
- [11] 徐亮. 基于改进 rrt* 和 q-learning 算法的机器人路径规划研究[D]. 吉林大学, 吉林, 2023.
- [12] Wei J, Wang Q, Zhao Z. Generative adversarial network based on Poincaré distance similarity constraint: focusing on overfitting problem caused by finite training data[J]. Applied Soft Computing, 2024, 151.
- [13] 孙瑞波, 王克, 高策. 机构化环境中基于 a* 算法的机器人路径规划算法的研究[C]. [出版地不详: 出版者不详], 2019.